PostgreSQL 18

Índices en PostgreSQL

Descubre por qué tu consulta tarda 8 segundos en producción, léelo en el plan de EXPLAIN ANALYZE y bájala a milisegundos con el índice correcto.



Chamba Lab · 2026



Un índice es un atajo ordenado a tus filas

Como el índice de un libro: buscas y saltas a la página.

- Guarda los valores de una columna, ordenados, y dónde está cada fila
- Sin él, PostgreSQL lee la tabla entera, fila por fila
- El ejemplo de todo el manual: `pedidos` de una app de delivery en Lima, con 2 millones de filas

Anatomía: un CREATE INDEX por partes

```
CREATE INDEX idx_pedidos_cliente ON pedidos
(cliente_id, creado_en DESC)
```

- `CREATE INDEX` — la orden
- `idx_pedidos_cliente` — nombre
- `ON pedidos` — tabla
- `(cliente_id,` — 1.ª columna
- `creado_en DESC)` — 2.ª columna y orden

Nombre

Libre. Usa `idx_tabla_columna`: es lo que verás en el plan.

Tabla

Un índice pertenece a una sola tabla.

Columnas

El orden importa: primero lo que filtras, luego lo que ordenas.

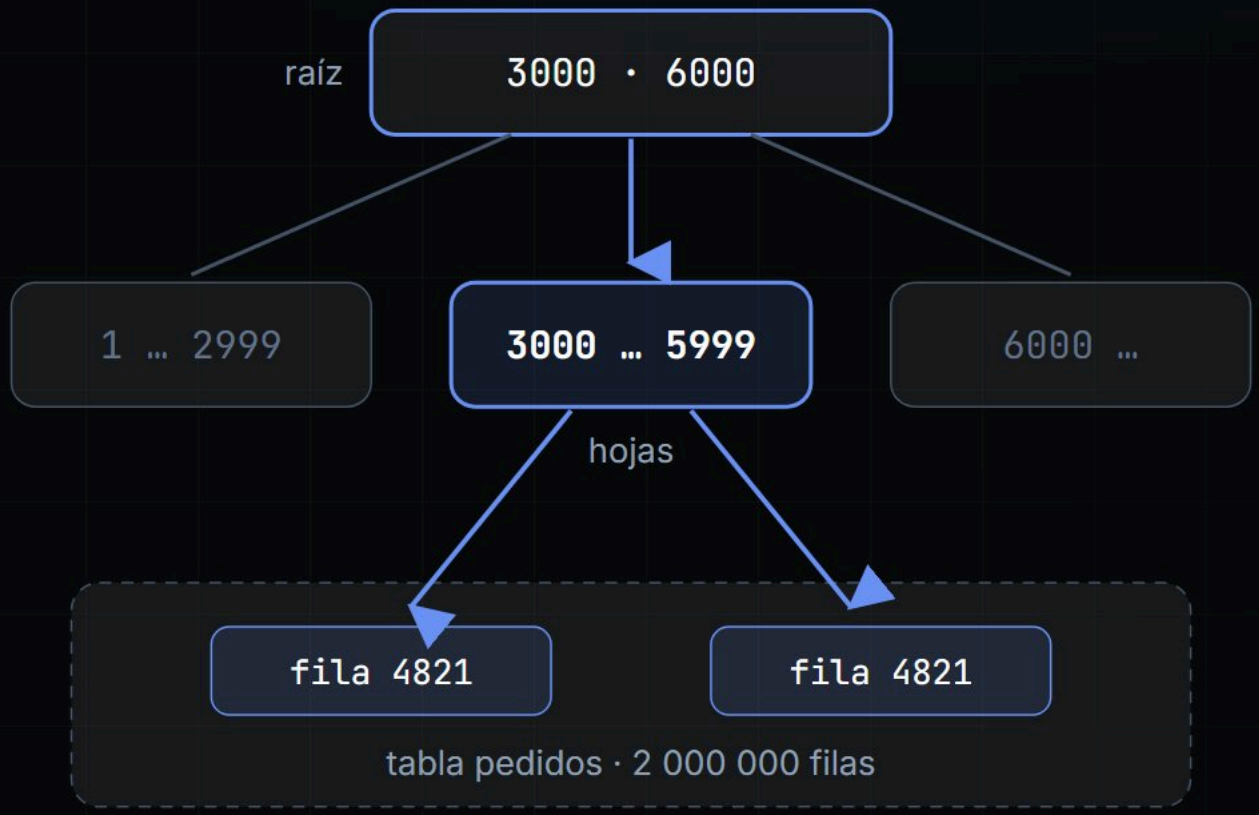
Método

Sin `USING` es un B-tree. Otros: `USING gin`, `brin`.

Cómo funciona: el B-tree baja por una sola rama

1. Empieza en la raíz y compara: 4821 cae entre 3000 y 5999
2. Baja por esa rama; las demás ni se leen
3. En la hoja, todas las entradas 4821 están juntas
4. Cada entrada dice dónde está la fila en la tabla

Más del 99 % de las páginas de un B-tree son hojas: unos pocos saltos cubren millones de filas.



Tipos: B-tree para casi todo

B-tree

Igualdad y rangos sobre valores que se pueden ordenar. También da el orden de `ORDER BY`.

`=` `< >` `IN` `LIKE 'ab%'`

POR DEFECTO

GIN

Valores con muchas partes: busca qué filas contienen un elemento.

`arrays` `jsonb` `pg_trgm`

FRECUENTE

BRIN

Resume bloques de la tabla. Diminuto, para tablas enormes que crecen en orden.

`logs por fecha`

TABLAS GIGANTES

GiST · Hash

GiST: geometría y vecino más cercano (PostGIS). Hash: solo igualdad.

`mapas` `=`

CASOS CONCRETOS

Quién decide: el planificador y sus fuentes

Estadísticas

`ANALYZE`

Cuántas filas hay y cómo se reparten los valores

Índice

`idx_pedidos_cliente`

Dónde está cada valor, en orden

Planificador

Tabla (heap)

`pedidos`

Las filas completas, en páginas de 8 kB

Mapa de visibilidad

`Index Only Scan`

Si una página es visible para todos, ni la abre

Estima el coste de cada camino y elige el más barato. Si las estadísticas mienten, elige mal.



La hámster QA: corre mucho y siempre llega al final

Seq Scan: leer la tabla entera

Sin índice útil, recorre todas las páginas.

- Revisa cada fila y descarta las que no cumplen el `WHERE`
- En el plan lo delata `Rows Removed by Filter`
- No es un error: si vas a traer gran parte de la tabla, es lo más rápido
- Con 100 filas, todo cabe en una página: ningún índice le gana

Con índice: tres formas de leer

`Index Scan`

Pocas filas

Recorre el índice y va a la tabla por cada fila, ya en orden. El típico de `WHERE id = 42`.

UNA O POCAS FILAS

`Bitmap Heap Scan`

Una cantidad media

Primero junta en el índice dónde están todas las filas y luego las trae de la tabla. Verás `Recheck Cond`.

CIENTOS O MILES

`Index Only Scan`

Sin tocar la tabla

Si el índice tiene todas las columnas que pides, responde solo. `INCLUDE (total)` las añade.

SOLO COLUMNAS DEL ÍNDICE



Rakim y Bunny: buscando a oscuras en producción

En local, 2 ms. En producción, 8 segundos.

La misma consulta y el mismo PostgreSQL.

- La pantalla «Mis pedidos» se queda cargando
- Soporte se llena de quejas y nadie tocó ese código
- En tu laptop hay 100 pedidos; en producción, 2 millones

El dolor: lo probaste con 100 filas

Nadie indexó `cliente_id`. La PK trae su índice; la FK, no. Con 100 filas da igual: caben en una página. Con 2 millones, cada visita lee la tabla entera.

- «Es especialmente fatal usar datos de prueba muy pequeños» (doc de PostgreSQL)

```sql
-- la migración
CREATE TABLE pedidos (
  id          bigint GENERATED ALWAYS AS IDENTITY
              PRIMARY KEY,
  cliente_id  bigint NOT NULL REFERENCES clientes,
  estado      text NOT NULL,
  total       numeric(10,2) NOT NULL,
  creado_en   timestamptz NOT NULL DEFAULT now()
);

-- la pantalla «Mis pedidos»
SELECT * FROM pedidos
WHERE cliente_id = 4821
ORDER BY creado_en DESC LIMIT 20;
```

Léelo en el plan: EXPLAIN ANALYZE

psql · producción (recortado)

```
EXPLAIN ANALYZE SELECT * FROM pedidos
WHERE cliente_id = 4821
ORDER BY creado_en DESC LIMIT 20;

Limit (actual time=8012.4..8012.5 rows=20.00 loops=1)
 → Sort (actual time=8012.4..8012.4 rows=20.00 loops=1)
     Sort Key: creado_en DESC
     → Seq Scan on pedidos
          (cost=0.00..48920.00 rows=37 width=52)
          (actual time=4.1..8011.8 rows=37.00 loops=1)
        Filter: (cliente_id = 4821)
        Rows Removed by Filter: 1999963
Execution Time: 8012.9 ms
```

`cost` es una estimación en unidades propias; `actual time`, milisegundos reales.

`rows=37` estimadas frente a `37.00` reales: si difieren mucho, corre `ANALYZE`.

`Seq Scan` + `Rows Removed by Filter`: leyó 2 millones para quedarse con 37.

Ojo: `ANALYZE` ejecuta la consulta. Un `UPDATE`, dentro de `BEGIN … ROLLBACK`.

El arreglo: el índice que pide la consulta

- Filtra por `cliente_id` y ordena por `creado_en`: el índice ya entrega las filas en ese orden, sin `Sort`
- Con `LIMIT 20` lee 20 entradas y se detiene
- `CONCURRENTLY` no bloquea las escrituras mientras se construye; tarda más y no va dentro de una transacción
- Si falla, queda un índice `INVALID`: bórralo y repite

```
CREATE INDEX CONCURRENTLY idx_pedidos_cliente
  ON pedidos (cliente_id, creado_en DESC);

-- el mismo EXPLAIN ANALYZE, otra vez:
-- Limit (actual time=0.04..0.09 rows=20.00)
--   → Index Scan using idx_pedidos_cliente
--       on pedidos
--       Index Cond: (cliente_id = 4821)
-- Execution Time: 0.12 ms
```

Índice compuesto: el orden de columnas manda



- Igualdad primero; rango u orden al final
- Sin la 1.ª columna, octubre queda repartido entre todos los clientes
- PostgreSQL 18 añade *skip scan*: a veces lo usa igual, si la 1.ª columna tiene pocos valores distintos

```
-- idx_pedidos_cliente
--   (cliente_id, creado_en DESC)

-- sí: igualdad + orden
WHERE cliente_id = 4821
ORDER BY creado_en DESC

-- sí: igualdad + rango
WHERE cliente_id = 4821
  AND creado_en ⩾ '2026-10-01'

-- no: falta la 1.ª columna
WHERE creado_en ⩾ '2026-10-01'
```

Hay índice y no lo usa: cuatro causas

Una función sobre la columna

```
WHERE lower(email) = 'ana@correo.pe'
```

El índice guarda `email`, no `lower(email)`.

```
CREATE INDEX ON clientes (lower(email));
```

Comodín al inicio

```
WHERE nombre LIKE '%quispe%'
```

B-tree solo sirve con `'quispe%'` (y `text_pattern_ops` si la base no usa locale C).

```
USING gin (nombre gin_trgm_ops)
```

Baja selectividad

```
WHERE estado = 'entregado' -- 90 %
```

Saltar del índice a la tabla 1,8 millones de veces cuesta más que leerla. Seq Scan es correcto.

```
… ON pedidos (creado_en) WHERE estado =
'pendiente'
```

Estadísticas viejas

```
rows=40 estimadas · rows=200000.00 reales
```

Tras una carga masiva, el planificador aún cree que la tabla es pequeña.

```
ANALYZE pedidos;
```



Demasiadas pestañas: cada una abierta cuesta algo

Cada índice se paga al escribir

Acelera lecturas; cada escritura lo actualiza.

- Un `INSERT` en `pedidos` escribe en la tabla y en cada uno de sus índices
- Ocupan disco y pueden impedir las actualizaciones HOT, las más baratas
- Un `CREATE INDEX` normal bloquea escrituras: en producción, `CONCURRENTLY`
- ¿Sobra alguno? Busca `idx_scan = 0` en `pg_stat_user_indexes`

Matriz de decisión: qué índice crear

	Sin índice	B-tree	GIN + pg_trgm
Igualdad o rango: `cliente_id = 4821`	Lee toda la tabla	Su caso	No aplica
Filtrar y ordenar con `LIMIT`	Lee todo y ordena	Compuesto: lee solo 20	No da orden
Texto en medio: `LIKE '%quispe%'`	Lee toda la tabla	No sirve	Su caso
Traes gran parte de la tabla	Lo más rápido	Lo ignora	Lo ignora
Coste al escribir	Ninguno	Moderado	El más alto

Mide con EXPLAIN ANALYZE sobre datos reales. Para cada consulta frecuente que trae pocas filas, un B-tree (igualdad → rango u orden). GIN solo para buscar dentro de texto, arrays o jsonb.

Lo que te llevas

- Mide con EXPLAIN ANALYZE y volumen real, nunca con 100 filas
- Igualdad primero, rango u orden después; la FK no crea índice
- Cada índice cuesta al escribir: créalo CONCURRENTLY y borra los que nadie usa

> docs: postgresql.org/docs/18/indexes.html

> docs: postgresql.org/docs/18/using-explain.html

> docs: postgresql.org/docs/18/indexes-multicolumn.html

> repo: github.com/Chamba-Lab